

Resumo Completo: Capítulo 3 - Gerenciamento de Memória

Introdução ao Gerenciamento de Memória

A memória principal (RAM) é um recurso essencial e finito que precisa ser gerenciado cuidadosamente pelo sistema operacional. Embora a capacidade de memória tenha crescido exponencialmente, os programas também cresceram, seguindo a "Lei de Parkinson": *“programas se expandem para preencher a memória disponível”*.

O objetivo de um programador seria uma memória ideal: infinitamente grande, extremamente rápida, barata e não volátil (que não perde dados sem energia). Como isso não existe, a solução prática é a **hierarquia de memórias**:

1. **Caches:** Pequenos, muito rápidos e caros (SRAM).
2. **Memória Principal:** Média, velocidade média e custo médio (DRAM).
3. **Armazenamento Secundário:** Grande, lento e barato (SSD, HD).

É função do Sistema Operacional (SO) abstrair essa hierarquia complexa em um modelo simples e eficiente para os processos. A parte do SO responsável por isso é o **Gerenciador de Memória**.

Funções do Gerenciador de Memória:

- Controlar quais partes da memória estão em uso.
- Alocar memória para processos quando necessário.
- Liberar a memória quando os processos terminam.
- Gerenciar a troca de dados entre a memória principal e o disco.

3.1 Sem Abstração de Memória

Os primeiros sistemas computacionais não ofereciam nenhuma abstração de memória. O programa via e acessava diretamente a memória física.

- **Modelo de Memória:** O programa via um único espaço de endereços físicos, de 0 até um máximo. Uma instrução como `MOV REGISTER1, 1000` acessava diretamente o endereço físico 1000.
- **Problemas:**
 1. **Ausência de Multitarefa Real:** Apenas um programa podia estar na memória por vez. Se dois programas estivessem na memória, um poderia facilmente sobrescrever os dados do outro, causando falhas catastróficas.
 2. **Vulnerabilidade do SO:** Um programa de usuário malicioso ou com bugs poderia sobrescrever e derrubar o sistema operacional.

Organização da Memória sem Abstração

O SO e o programa do usuário precisavam coexistir na memória. As três principais configurações eram:

1. **SO na RAM baixa:** O SO reside nos endereços mais baixos da memória.
2. **SO na ROM alta:** O SO reside em memória somente leitura (ROM) nos endereços mais altos, protegendo-o de ser sobrescrito. Comum em sistemas embarcados.

3. **Drivers em ROM, resto do SO em RAM:** Um modelo híbrido, onde a BIOS (Basic Input Output System) ficava em ROM e o resto do SO era carregado na RAM.

Soluções Primitivas para Multiprogramação

Mesmo sem abstração, algumas técnicas permitiam a execução de múltiplos programas:

1. **Swapping (Troca de Processos):** O SO salva o conteúdo inteiro da memória em disco, carrega o próximo programa, o executa por um tempo e repete o processo. Garante que apenas um programa ocupe a memória por vez.
2. **Proteção por Hardware (Ex: IBM 360):**
 - A memória era dividida em blocos de 2 KB.
 - Cada bloco tinha uma **chave de proteção** de 4 bits.
 - A CPU continha um registrador especial (PSW - Program Status Word) que também tinha uma chave de 4 bits.
 - O hardware impedia que um processo acessasse um bloco de memória cuja chave de proteção fosse diferente da chave em sua PSW.
 - **Problema Resolvido:** Proteção. Um processo não pode interferir com outro ou com o SO.
 - **Problema Não Resolvido: Realocação.** Se dois programas fossem carregados sequencialmente, as referências a endereços absolutos (ex: `JMP 28`) estariam erradas para o segundo programa, que não foi carregado no endereço 0.
3. **Realocação Estática:**
 - **Solução:** O *carregador* (loader) modifica os endereços no código do programa à medida que ele é carregado na memória. Por exemplo, se o programa é carregado no endereço 16384, toda referência a um endereço `X` é convertida para `16384 + X`.
 - **Desvantagens:** É lenta e complexa, pois o carregador precisa saber distinguir o que é um endereço (que deve ser realocado) de uma constante (que não deve).

3.2 Uma Abstração de Memória: Espaços de Endereçamento

A solução moderna para os problemas de proteção e realocação é criar uma abstração chamada **espaço de endereçamento** (*address space*).

- **Conceito:** Cada processo tem seu próprio espaço de endereçamento privado, que é um conjunto de endereços que ele pode usar. Os endereços nesse espaço são independentes dos endereços físicos.
- **Endereços Virtuais vs. Físicos:**
 - **Endereço Virtual:** Endereço gerado pela CPU, dentro do espaço de endereçamento do processo.
 - **Endereço Físico:** Endereço real na memória principal.
- O hardware (MMU) traduz dinamicamente os endereços virtuais em endereços físicos durante a execução.

Realocação Dinâmica com Registradores Base e Limite

Uma forma simples de implementar espaços de endereçamento:

- **Hardware Necessário:** Dois registradores na CPU:
 1. **Registrador Base:** Armazena o endereço físico inicial onde o processo está carregado.
 2. **Registrador Limite:** Armazena o tamanho do processo.
- **Funcionamento:**
 1. Quando um processo é executado, o SO carrega seus registradores base e limite.
 2. Para cada endereço virtual gerado pela CPU, o hardware:
 - **Soma** o endereço virtual ao valor do **registrador base** para obter o endereço físico.
 - **Compara** o endereço virtual com o **registrador limite**. Se `endereço_virtual >= limite`, ocorre uma falha (trap), pois o processo está tentando acessar memória fora de seu espaço.
- **Vantagens:** Soluciona tanto a proteção quanto a realocação de forma dinâmica e simples.
- **Desvantagens:** Requer uma soma e uma comparação em *toda* referência à memória, o que pode introduzir lentidão.

Gerenciando a Memória Livre

Com o *swapping* e a alocação dinâmica, a memória se fragmenta em blocos ocupados e livres ("buracos"). O SO precisa gerenciar esses buracos.

Métodos de Gerenciamento:

1. Mapas de Bits (Bitmaps):

- A memória é dividida em unidades de alocação.
- O SO mantém um vetor de bits, com um bit para cada unidade (ex: `1` = ocupado, `0` = livre).
- **Vantagem:** Simples.
- **Desvantagem:** Procurar um bloco contíguo de `k` unidades livres (`k` bits `0`) pode ser uma operação lenta.

2. Listas Encadeadas:

- O SO mantém uma lista encadeada de blocos (segmentos) alocados e livres.
- Cada nó da lista contém: tipo (processo ou buraco), endereço inicial, tamanho e ponteiro para o próximo.
- **Vantagem:** Facilita a fusão de buracos adjacentes quando um processo termina.

Algoritmos de Alocação de Memória (para listas encadeadas):

- **First Fit (Primeiro Encaixe):** Varre a lista desde o início e aloca o primeiro buraco que seja grande o suficiente.
 - **Vantagem:** Rápido.
 - **Desvantagem:** Tende a criar pequenos fragmentos inúteis no início da lista.
- **Next Fit (Próximo Encaixe):** Similar ao First Fit, mas começa a busca a partir de onde a última busca parou.
 - **Desempenho:** Ligeiramente pior que o First Fit.
- **Best Fit (Melhor Encaixe):** Varre a lista inteira e escolhe o menor buraco que seja grande o suficiente.
 - **Vantagem:** Evita desperdiçar buracos grandes.

- **Desvantagem:** Lento (sempre percorre a lista toda) e tende a criar fragmentos minúsculos e inúteis.
 - **Worst Fit (Pior Encaixe):** Varre a lista toda e escolhe o maior buraco. A ideia é que o fragmento restante seja grande e útil.
 - **Desempenho:** Simulações mostram que não é uma boa estratégia.
 - **Quick Fit:** Mantém listas separadas para tamanhos de buracos mais comuns.
 - **Vantagem:** Alocação extremamente rápida.
 - **Desvantagem:** A fusão de buracos é complexa.
-

3.3 Memória Virtual

A **memória virtual** é uma técnica que permite a um programa ser executado mesmo que apenas parte dele esteja na memória principal. Ela generaliza o conceito de espaço de endereçamento.

- **Motivação:**
 1. Permitir que programas sejam maiores que a memória física disponível.
 2. Melhorar a multiprogramação, mantendo mais processos "parcialmente" na memória.

3.3.1 Paginação (Paging)

É a técnica mais comum para implementar memória virtual.

- **Conceitos Fundamentais:**
 - **Página (Page):** Bloco de tamanho fixo de endereços virtuais.
 - **Quadro de Página (Page Frame):** Bloco de tamanho fixo de memória física.
 - **Tamanho:** O tamanho da página é sempre igual ao do quadro de página (ex: 4 KB).
- **Funcionamento:**
 - O espaço de endereçamento virtual de um processo é dividido em páginas.
 - A memória física é dividida em quadros de página.
 - A **MMU (Memory Management Unit)**, uma unidade de hardware, traduz os endereços virtuais em físicos.

3.3.2 Tabelas de Páginas (Page Tables)

A tradução de endereços é feita com base em uma **tabela de páginas** por processo.

- **Mecanismo de Tradução:**
 1. Um endereço virtual é dividido em duas partes: **Número da Página Virtual** e **Deslocamento (Offset)**.
 2. A MMU usa o *Número da Página Virtual* como um índice na tabela de páginas do processo.
 3. A entrada da tabela de páginas contém o **Número do Quadro de Página** correspondente na memória física.
 4. O endereço físico é formado pela concatenação do *Número do Quadro de Página* com o *Deslocamento*.
- **Falta de Página (Page Fault):**
 - Ocorre quando um processo tenta acessar uma página que não está na memória física.

- A entrada na tabela de páginas possui um **bit Presente/Ausente**. Se este bit for "ausente", a MMU gera uma interrupção (trap) para o SO.
- **Tratamento pelo SO:**
 1. O SO localiza a página necessária no disco.
 2. Encontra um quadro de página livre na RAM. Se não houver, executa um *algoritmo de substituição de página* para liberar um.
 3. Carrega a página do disco para o quadro livre.
 4. Atualiza a tabela de páginas para mapear a página virtual para o novo quadro físico.
 5. Reinicia a instrução que causou a falta.

- **Estrutura de uma Entrada da Tabela de Páginas:**

- **Número do Quadro de Página:** O campo mais importante.
- **Bit Presente/Ausente:** Indica se a página está na RAM.
- **Bits de Proteção:** Controlam o acesso (Leitura, Escrita, Execução).
- **Bit Modificada (Modified/Dirty):** O hardware o ativa quando a página é escrita. Se o bit estiver ativo, a página precisa ser salva no disco antes de ser substituída.
- **Bit Referenciada (Referenced):** O hardware o ativa sempre que a página é acessada (lida ou escrita). Usado por algoritmos de substituição.
- **Bit de Desabilitação de Cache:** Permite desativar o cache para páginas mapeadas em dispositivos de E/S.

3.3.3 Acelerando a Paginação: TLB (Translation Lookaside Buffer)

- **Problema:** A paginação exige um acesso extra à memória para cada referência (um para a tabela de páginas, outro para o dado/instrução), o que cortaria o desempenho pela metade.
- **Solução:** O **TLB**, também chamado de *memória associativa*, é um pequeno cache de hardware que armazena as entradas da tabela de páginas usadas mais recentemente.
- **Funcionamento:**
 1. Quando um endereço virtual é gerado, a MMU primeiro verifica se o número da página está no TLB.
 2. **TLB Hit (Acerto):** Se estiver, o número do quadro é obtido diretamente do TLB, sem acessar a tabela de páginas na memória. Rápido.
 3. **TLB Miss (Falta):** Se não estiver, a MMU realiza o acesso normal à tabela de páginas na memória, obtém a entrada, e então a armazena no TLB (substituindo uma entrada antiga se o TLB estiver cheio).

3.3.4 Tabelas de Páginas para Memórias Grandes

- **Problema:** Para um espaço de 32 bits com páginas de 4 KB, a tabela de páginas teria 1 milhão de entradas, ocupando 4 MB por processo. Para 64 bits, é impraticável.
- **Soluções:**
 1. **Tabelas de Páginas Multinível:**
 - O endereço virtual é dividido em múltiplos campos (ex: **Diretório**, **Tabela**, **Deslocamento**).
 - Cria-se uma hierarquia: uma tabela de páginas de nível superior aponta para tabelas de nível inferior.

- **Vantagem:** As tabelas de páginas de nível inferior só precisam existir se as páginas correspondentes estiverem em uso, economizando muito espaço.

2. Tabelas de Páginas Invertidas:

- Em vez de uma entrada por página virtual, há uma entrada por *quadro de página físico*.
- Cada entrada armazena qual par (**processo**, **página_virtual**) está ocupando aquele quadro.
- **Vantagem:** O tamanho da tabela depende da memória física, não do espaço virtual, economizando muito espaço em sistemas de 64 bits.
- **Desvantagem:** A tradução de endereço virtual para físico se torna uma busca na tabela. Para acelerar, usa-se uma tabela de hash.

3.4 Algoritmos de Substituição de Páginas

Quando ocorre uma falta de página e não há quadros livres, o SO deve escolher uma página para remover (evicção). Um bom algoritmo reduz a taxa de faltas de página.

- **Algoritmo Ótimo:** Remove a página que será usada mais distante no futuro.
 - **Desempenho:** Perfeito.
 - **Implementação:** Impossível, pois exige prever o futuro. É usado apenas como um padrão de comparação (benchmark).
- **NRU (Not Recently Used - Não Usado Recentemente):**
 - Usa os bits *Referenciada (R)* e *Modificada (M)*.
 - Classifica as páginas em 4 classes:
 - Classe 0: não referenciada, não modificada ($\neg R$, $\neg M$)
 - Classe 1: não referenciada, modificada ($\neg R$, M)
 - Classe 2: referenciada, não modificada (R , $\neg M$)
 - Classe 3: referenciada, modificada (R , M)
 - Remove uma página aleatória da classe mais baixa que não esteja vazia.
- **FIFO (First-In, First-Out):** Remove a página que está na memória há mais tempo.
 - **Implementação:** Simples (uma fila).
 - **Desempenho:** Ruim, pois pode remover uma página antiga que é intensamente usada.
- **Segunda Chance:** Uma melhoria do FIFO.
 - Inspecciona o bit R da página mais antiga. Se $R=0$, a remove. Se $R=1$, zera o bit R, move a página para o fim da fila (como se tivesse acabado de chegar) e continua a busca.
- **Relógio:** Uma implementação mais eficiente da Segunda Chance.
 - Os quadros de página são mantidos em uma lista circular (como um relógio).
 - Um ponteiro ("ponteiro do relógio") aponta para a página mais antiga.
 - Em uma falta de página, o algoritmo inspeciona a página apontada. Se $R=0$, a remove. Se $R=1$, zera o bit R e avança o ponteiro. Repete até encontrar uma página com $R=0$.
- **LRU (Least Recently Used - Menos Recentemente Usado):**
 - Remove a página que não é usada há mais tempo.
 - **Desempenho:** Excelente, pois se aproxima bem do ótimo.
 - **Implementação:** Difícil e cara. Exigiria hardware para registrar o tempo de cada acesso ou manter uma lista ordenada por uso.
- **NFU (Not Frequently Used) / Envelhecimento (Aging):** Simulação de LRU em software.
 - Cada página tem um contador.

- Periodicamente, o SO desloca o contador de cada página para a direita e adiciona o bit R na posição mais à esquerda.
 - A página com o menor valor no contador é a candidata à remoção.
 - **Conjunto de Trabalho (Working Set):**
 - Baseado no princípio da **localidade de referência**. O conjunto de trabalho de um processo é o conjunto de páginas que ele referenciou recentemente.
 - O algoritmo tenta manter o conjunto de trabalho de um processo na memória. Remove uma página que não está no conjunto de trabalho atual.
 - **WSClock:** Combina a eficiência do Relógio com a lógica do Conjunto de Trabalho.
 - Usa uma lista circular. Quando o ponteiro encontra uma página, verifica seu bit R. Se R=1, a página está em uso e seu "tempo de último uso" é atualizado. Se R=0, o algoritmo calcula a "idade" da página. Se for maior que um limiar (τ), a página não está mais no conjunto de trabalho e é removida.
 - **Desempenho:** Bom e eficiente de implementar.
-

3.5 Questões de Projeto para Sistemas de Paginação

- **Políticas de Alocação (Local vs. Global):**
 - **Local:** Um processo só pode substituir suas próprias páginas.
 - **Global:** Um processo pode pegar um quadro de página de qualquer outro processo.
 - **Comparação:** Políticas globais geralmente têm melhor desempenho, pois alocam memória de forma mais flexível.
 - **Controle de Carga e Thrashing:**
 - **Thrashing:** Ocorre quando a soma dos conjuntos de trabalho de todos os processos excede a memória física. O sistema passa mais tempo paginando do que executando.
 - **Solução:** O SO deve detectar o *thrashing* e reduzir o nível de multiprogramação, suspendendo (fazendo *swap out*) um ou mais processos para liberar quadros de página.
 - **Tamanho da Página:**
 - **Páginas Pequenas:** Menor fragmentação interna, mas tabelas de páginas maiores.
 - **Páginas Grandes:** Tabelas menores, E/S mais eficiente, mas maior fragmentação interna.
 - O valor típico hoje é 4 KB ou 8 KB, um equilíbrio entre esses fatores.
 - **Páginas Compartilhadas:**
 - Para economizar memória, múltiplos processos podem compartilhar páginas (ex: bibliotecas de código).
 - **Copy-on-Write (COW):** Uma técnica eficiente para compartilhar dados. Dois processos compartilham a mesma página em modo "somente leitura". Se um deles tenta escrever, o SO intercepta a operação, cria uma cópia privada da página para aquele processo e então permite a escrita.
-

3.7 Segmentação

A segmentação é outra forma de memória virtual onde o espaço de endereçamento não é linear, mas sim uma coleção de **segmentos** de tamanhos variáveis.

- **Endereçamento:** Um endereço é um par (**número_do_segmento**, **deslocamento**).
- **Vantagens:**

- **Visão Lógica:** Corresponde à visão do programador (código, dados, pilha são segmentos distintos).
- **Facilita Proteção e Compartilhamento:** É fácil aplicar diferentes proteções (leitura, escrita, execução) a segmentos diferentes ou compartilhar um segmento de código entre processos.
- **Gerenciamento de Estruturas Dinâmicas:** Pilhas e heaps podem crescer e encolher sem interferir um com o outro.
- **Desvantagem:** Causa **fragmentação externa** (buracos de memória entre os segmentos), que exige compactação (operação cara) para ser resolvida.

Segmentação com Paginação (Ex: MULTICS e Intel x86)

Combina as vantagens de ambos os mundos: a visão lógica da segmentação com a gestão de memória eficiente da paginação.

- **Funcionamento:**
 1. O SO divide o espaço de endereçamento em segmentos.
 2. Cada segmento, por sua vez, é dividido em páginas de tamanho fixo.
- **Tradução de Endereço (Ex: Intel x86-32):**
 1. Um endereço lógico (**seletor**, **deslocamento**) é gerado.
 2. O **seletor** aponta para um **descritor de segmento** em uma tabela (GDT ou LDT).
 3. O descritor contém o endereço base e o limite do segmento. O endereço base é somado ao deslocamento para formar um **endereço linear** de 32 bits.
 4. Se a paginação estiver ativa, esse endereço linear é então tratado como um endereço virtual e traduzido para um endereço físico através das tabelas de páginas multinível.
- **Declínio:** Embora poderosa, a segmentação complexa (como no x86-32) não foi amplamente utilizada por sistemas como UNIX e Windows, que preferiram um modelo de memória virtual paginado mais simples ("plano"). Por isso, no x86-64, a segmentação foi largamente abandonada.